

01 · AND · OR · XOR 논리 게이트

VS Code 사전 시뮬레이션 → 레거시 Vivado → 보드 실험

같은 입력 a, b에 대해 x는 AND, y는 OR, z는 XOR이다.

1. 템플릿을 clone하고 이 회로의 workspace를 연다.
2. 예상값을 계산하고 RTL·TB·XDC를 직접 작성해 시뮬레이션한다.
3. Vivado 결과와 실제 보드 동작을 비교한다.
4. 자신의 사진·영상·레포트를 GitHub에 연결한다.

작성일 2026. 09. 10. · 이해리

실습 목차

1부 · VS Code 사전 실습

새 창·workspace·확장·코드 열기

예상값·작업 실행·로그·파형·실험 전 레포트

2부 · Vivado 실습

프로젝트·소스·top·시뮬레이션·핀·비트스트림

3부 · 결과 정리

보드·사진·영상·GitHub·실험 후 레포트

전체 목차 PDF 열기

시뮬레이션 도구 확인

Git·Python 3.10 이상·VS Code·Icarus Verilog를 설치한다. Windows PowerShell에서 한 줄씩 확인한다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

설치·PATH 변경 후 VS Code를 다시 연다. Linux·macOS·WSL은 python 대신 python3를 사용한다.

설치와 빈 템플릿 안내

v2.0.0 템플릿을 새 이름으로 clone

아래는 Windows PowerShell 명령이다. 줄 끝 문자는 다음 줄로 명령을 이어 준다.

```
git clone --branch v2.0.0 `
https://github.com/Glaysia/fpga-lab-template.git `
lab1_legacy_01_logic_gates
cd lab1_legacy_01_logic_gates
git switch -c main
code LAB1.code-workspace
```

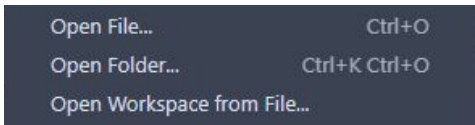
회로마다 마지막 폴더 이름을 바꾼다. 시작 파일은 주석뿐이며 코드 이미지를 보며 직접 작성한다.
고정된 수업 버전 v2.0.0

File → New Window

VS Code 상단 File → New Window. Ctrl+Shift+N도 같은 동작이다.



새 창의 File → Open Workspace from File...을 누른다.



프로젝트 하나의 workspace 열기

1. 방금 clone한 lab1_legacy_01_logic_gates 폴더를 연다.
2. LAB1.code-workspace를 선택하고 Open. 모든 실습의 workspace 파일명은 같다.
3. Explorer에 PROJECT 하나와 src.sim.constraints.tools 폴더가 보이는지 확인한다.
4. src/design.v.sim/tb_design.v.constraints/pins.xdc는 아직 주석뿐이다.
5. simulation.json은 실행할 RTL·TB 파일과 TB top을 지정하는 설정이다.

추천 확장 설치

1. 왼쪽 Extensions 아이콘을 누른다.
2. 검색창에 @recommended를 입력한다.
3. slang의 Install: Verilog/SystemVerilog 편집·진단.
4. VaporView의 Install: VCD 파형 확인.
5. vscode-pdf의 Install: 코드 옆에서 PDF 교안 열기.

WSL 사용자는 WSL 창에서 필요할 경우 Install in WSL을 누른다. 확장 설치와 Python·Icarus 설치의 별개다.

src에 RTL 파일 만들기

1. Explorer에서 src 왼쪽 화살표를 눌러 펼친다.
 2. design.v 우클릭 → Rename → logic_gate.v 입력.
 3. 파일을 두 번 눌러 열고 뒤의 코드 이미지를 보며 전체 코드를 입력한다.
 4. 추가 RTL이 있으면 src 우클릭 → New File로 만든다.
 5. 전체 RTL 파일 목록은 다음 장의 src 표를 따른다.
 6. File → Save All로 모든 파일을 저장한다.
- 설계 모듈명: logic_gate. 파일명과 module 이름을 구분한다.

sim에 테스트벤치 만들기

1. sim을 펼치고 tb_design.v 우클릭 → Rename.
2. 새 파일명: tb_logic_gate_modern.sv
3. 뒤의 TB 코드 이미지에 있는 선언·DUT 연결·입력 자극·예상값
검사를 직접 작성한다.
4. wave.vcd를 만드는 dumpfile·dumpvars와 종료 조건까지 작성한다.
5. TB 모듈명: tb_logic_gate_modern
6. File → Save All. TB를 합성용 RTL 폴더에 넣지 않는다.

작성할 RTL 파일 목록

폴더	파일명
src	logic_gate.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

simulation.json을 내 파일에 맞추기

Explorer에서 simulation.json을 두 번 눌러 연다.

sources 배열: 앞의 모든 RTL 파일명 앞에 src/를 붙여 등록한다.

한 파일 예: "sources": ["src/logic_gate.v"]

testbench: sim/tb_logic_gate_modern.sv

simulation_top: tb_logic_gate_modern

sources의 각 경로와 testbench·simulation_top 값은 큰따옴표로 감싼다.
여러 경로는 쉼표로 구분한다.

파일명·확장자·괄호·쉼표를 확인하고 Save All.

VS Code 실습은 대응 최신판을 재사용

이 PDF의 사전 실습은 01번 최신판과 같은 RTL·자기검사 TB·실행 방법이다.

실습 폴더는 이번 레거시용 이름으로 만들되, 같은 코드 이미지를 보고 직접 작성한다.

VS Code의 PASS·파형·실험 전 레포트 뒤에 이 PDF의 원본 Vivado 강의자료를 진행한다.

원본 testbench.v는 자극 순서·시간이 다를 수 있다. 동일 결과 비교에는 앞에서 작성한 자기검사 TB를 등록한다.

01번 VS Code 상세 실습 열기

입출력과 동작 규칙

입력 a, b / 출력 x, y, z

같은 입력 a, b 에 대해 x 는 AND, y 는 OR, z 는 XOR이다.

KEY1= a , KEY2= b . LED1= x , LED2= y , LED3= z .

실행 전에 예상값 작성

입력 또는 조건	예상 결과
00	000
01	011
10	011
11	110

마지막 30–40ns 구간에서 AND와 OR는 1, XOR는 0이다.

예상값을 계산한 근거를 자신의 말로 설명한다.

논리 게이트 RTL 직접 작성

src/logic_gate.v의 전체 내용을 입력하고 저장한다.

```
1 module logic_gate(input wire a,b, output wire x,y,z);  
2     assign x = a & b;  
3     assign y = a | b;  
4     assign z = a ^ b;  
5 endmodule
```

같은 입력 a, b에 대해 x는 AND, y는 OR, z는 XOR이다.

테스트벤치와 통과 기준

시뮬레이션 top: tb_logic_gate_modern

4개 입력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 40ns다.

기대값과 실제 출력이 다르면 \$fatal로 중단한다. 검사 횟수와 watchdog도 확인한다.

통과 문구: LAB1_PASS logic_gate cases=4

원본 레거시 testbench.v와 별도의 자기검사 TB는 파일과 역할을 구분한다.

테스트벤치 직접 작성 · 1-12행

sim/tb_logic_gate_modern.sv · 같은 파일에 순서대로 이어 입력한다.

```
1 timescale 1ns/1ps
2 module tb_logic_gate_modern;
3     reg a,b; wire x,y,z; logic_gate dut(a,b,x,y,z);
4     integer n,checked=0;
5     reg [2:0] expected;
6     initial begin
7         $dumpfile("wave.vcd"); $dumpvars(0,tb_logic_gate_modern);
8
9         for(n=0;n<4;n=n+1) begin
10             {a,b}=n;
11             expected={(a & b),(a | b),(a ^ b)};
12             #10;
```

긴 오류 문자열은 화면 줄바꿈이다. 문자열 중간에 Enter를 넣지 않는다.

테스트벤치 직접 작성 · 13-22행

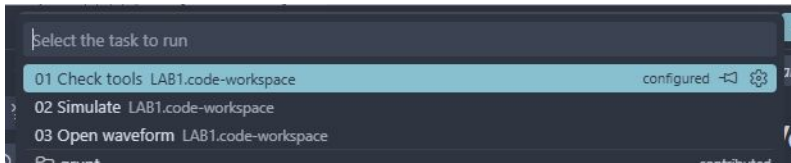
sim/tb_logic_gate_modern.sv · 같은 파일에 순서대로 이어 입력한다.

```
13         if ({x,y,z} != expected)
14             $fatal(1,"FAIL logic_gate vector=%0d expected=%h
    actual=%h",n,expected,{x,y,z});
15         checked=checked+1;
16     end
17     if(checked!=4) $fatal(1,"Incomplete test");
18     $display("LAB1_PASS logic_gate cases=%0d",checked);
19     $finish;
20 end
21 initial begin #100000; $fatal(1,"Watchdog"); end
22 endmodule
```

긴 오류 문자열은 화면 줄바꿈이다. 문자열 중간에 Enter를 넣지 않는다.

저장 → Terminal → Run Task

File → Save All 다음 Terminal → Run Task...을 누른다.



1. 01 Check tools를 실행하고 도구 버전을 확인한다.
2. 02 Simulate를 선택하고 종료까지 기다린다.
3. 03 Open waveform으로 생성된 VCD를 연다.

작업이 없으면 올바른 .code-workspace를 열었는지 확인한다.

실행 로그 확인

터미널과 `build/sim/run-.../simulation.log`에서 이번 실행을 확인한다.

`LAB1_PASS logic_gate cases=4`

정상 종료 시각: 40ns. PASS 문구와 검사 수를 함께 확인한다.

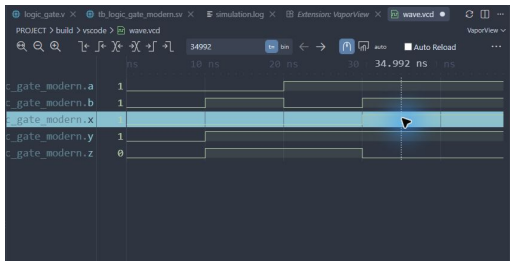
실패하면 이번 실행 폴더의 `compile.log`와 `simulation.log`에서 첫 오류를 찾는다.

코드 저장 → 02 Simulate 재실행 → 새 로그 확인 순서로 복귀한다.
프로젝트 검증 안내

VCD를 파형으로 열기

1. 03 Open waveform 또는 build/sim/wave.vcd를 연다.
2. 텍스트로 열리면 탭 우클릭 → Reopen Editor With... → VaporView.
3. Netlist View에서 tb_logic_gate_modern을 펼친다.
4. 신호 a, b, x, y, z를 각각 두 번 눌러 추가한다.
5. Zoom to Fit를 누르고 Time Units를 ns로 맞춘다.
6. 전환 경계 대신 구간 중간에 커서를 두고 값을 읽는다.

파형 화면의 읽는 순서



위는 공통 파형 조작 예다. 이번 회로에서는 앞 장의 신호 이름을 선택한다.

마지막 30–40ns 구간에서 AND와 OR는 1, XOR는 0이다.

신호 이름 → 시간 구간 → 커서 값 → 예상 결과 순서로 비교한다.

실제 VCD의 구간 중간값

시간(ns)	a	b	x	y	z
5	0	0	0	0	0
15	0	1	0	1	1
25	1	0	0	1	1
35	1	1	1	1	0

이 표는 실행된 VCD에서 읽은 값이다. 다중 비트는 이진수다.

표의 시간으로 파형 커서를 옮겨 직접 대조하고, 추가 경계 조건도 확인한다.

constraints에 XDC 직접 작성

1. constraints를 펼치고 pins.xdc 우클릭 → Rename.
2. 파일명: logic_gate.xdc
3. 핀 표와 코드 이미지를 보며 PACKAGE_PIN·IOSTANDARD·get_ports를 입력한다.
4. get_ports의 이름과 비트 번호를 자신이 작성한 RTL의 포트와 대조한다.
5. File → Save All. Vivado 또는 CLI 구현에는 이 파일을 직접 가져간다.

XDC는 Icarus 기능 시뮬레이션에서 사용하지 않는다. 파형 통과만으로 핀 배치까지 검증됐다고 판단하지 않는다.

XDC 전체 코드 · 1-12행

constraints/logic_gate.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
1  # Derived from vendor archive; see legacy provenance.
2  set_property PACKAGE_PIN N8 [get_ports b]
3  set_property PACKAGE_PIN L4 [get_ports x]
4  set_property PACKAGE_PIN M4 [get_ports y]
5  set_property PACKAGE_PIN M2 [get_ports z]
6  set_property IOSTANDARD LVCMOS33 [get_ports b]
7  set_property IOSTANDARD LVCMOS33 [get_ports a]
8  set_property IOSTANDARD LVCMOS33 [get_ports x]
9  set_property IOSTANDARD LVCMOS33 [get_ports y]
10 set_property IOSTANDARD LVCMOS33 [get_ports z]
11
12 set_property PACKAGE_PIN K4 [get_ports a]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

실험 전 레포트

1. 목적·포트·비트 폭·예상표와 동작 원리를 설명한다.
 2. 소스와 TB 링크, 코드 커밋, 도구 버전을 남긴다.
 3. 입력 조합·기대값·자동 비교·종료 조건을 설명한다.
 4. 자신의 PASS 로그와 파형 원본·캡처를 첨부한다.
 5. 시간 구간별 값을 해석하고 예상값과 비교한다.
 6. 오류·수정·재실행, 보드에서 확인할 입력과 출력을 적는다.
- 교재 캡처를 자신의 실행 결과로 제출하지 않는다.

원문 Vivado 실습으로 이동

이후 원문은 원본의 사진·문구·순서를 유지한다.

배포 프로젝트는 Vivado 2020.1 원본 XPR 형식을 기반으로 상대경로를 정리했다. 구버전 실행 검증은 별도로 확인한다.

수업에서는 원문의 합성·구현에 앞서 자기검사 TB로 Behavioral Simulation을 먼저 실행한다.

원본 testbench.v와 검증용 TB의 입력 순서·실행 시간은 서로 다를 수 있다.

레거시 01 · 논리 게이트

[실습 1] AND, OR, XOR 게이트 구현

- 입력 a, b에 대한 출력 x, y, z를 확인한다.
- 원본의 프로젝트 이름: logic_gate.
- 대상 부품: Spartan-7 xc7s75fpga484-1.

실습 순서

VS Code 사전 시뮬레이션·실험 전 레포트

Vivado 실습·FPGA 기록·동작 확인

사진·영상·GitHub·실험 후 레포트

작성일 2026. 09. 10. 이해리

원자료: 박동욱 교수, 서울시립대학교, 「논리 게이트 구현」.

다음 원문 페이지는 2022년 보관 PDF의 사진과 설명을 재구성했다.

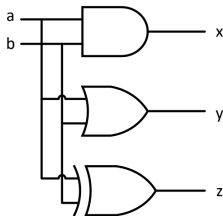
레거시 01 · Vivado 목차

설계 기획	블록도·진리표
(1) 프로젝트 선언	부품·경로·프로젝트 설정
(2) 로직 설계	원본 코드·소스 등록
(3) Assignment	입출력 핀·전기 규격
(4) Compile	합성·배치배선
(5) Simulation	테스트벤치·파형
(6) Programming	비트스트림·실제 장치 기록
(7) 동작 확인	스위치 입력·LED 출력

원본 순서를 보존했다. 수업에서는 합성·구현에 앞서 VS Code와 Vivado에서 시뮬레이션한다. [원문 보충 설명](#) · [실험 전·후 레포트](#) · [자료 안내](#)

설계 기획

블록도 및 진리표



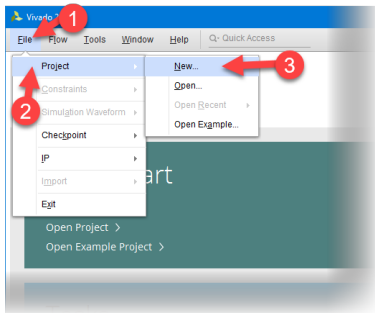
서울시립대학교 전자전기컴퓨터공학부

입력		출력		
a	b	AND x	OR y	XOR z
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

로직 설계 및 합성

(1) 프로젝트 선언

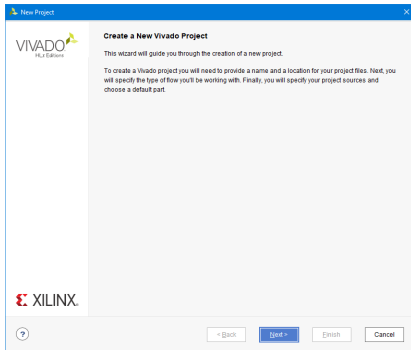
- Vivado 실행
- File > Project > New



로직 설계 및 합성

(1) 프로젝트 선언

- Next



로직 설계 및 합성

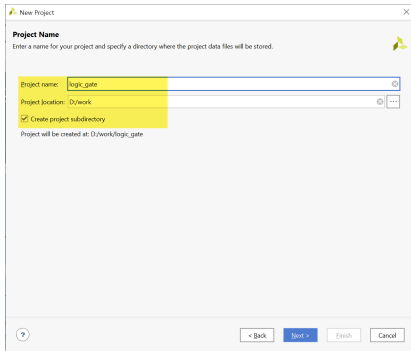
(1) 프로젝트 선언

- Project Name과 Project Location 설정
- Project Name : logic_gate
- Project location : d:/work
- Create project subdirectory : 체크
- d:/work 폴더에 logic_gate 폴더를 생성

로직 설계 및 합성

(1) 프로젝트 선언

원문 설명에 대응하는 실제 화면



A screenshot of a 'New Project' dialog box. The dialog has a title bar with a yellow icon and a close button. Below the title bar, it says 'Project Name' and 'Enter a name for your project and specify a directory where the project data files will be stored.' There are three input fields: 'Project name:' with the text 'logic_gate', 'Project location:' with the text 'D:/work', and a checked checkbox 'Create project subdirectory'. Below these fields, it says 'Project will be created at: D:/work/logic_gate'. At the bottom, there are four buttons: '< Back', 'Next >', 'Finish', and 'Cancel'.

New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: logic_gate

Project location: D:/work

☒ Create project subdirectory

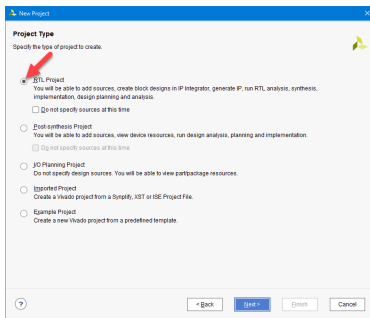
Project will be created at: D:/work/logic_gate

< Back Next > Finish Cancel

로직 설계 및 합성

(1) 프로젝트 선언

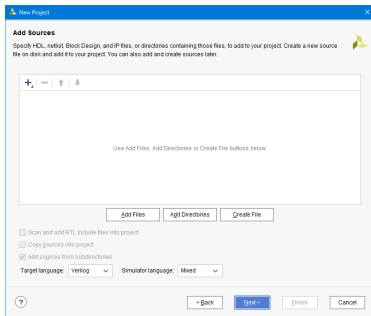
- Project Type 설정
- RTL Project



로직 설계 및 합성

(1) 프로젝트 선언

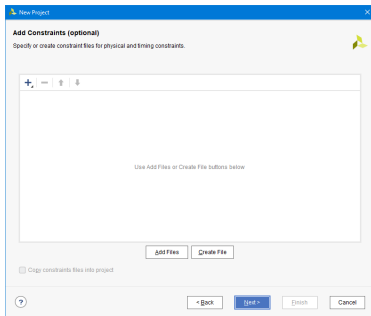
- Add source와 Add Constraints 설정
- 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



로직 설계 및 합성

(1) 프로젝트 선언

- Add source와 Add Constraints 설정
- 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



로직 설계 및 합성

(1) 프로젝트 선언

- Device 설정
- Family : Spartan-7
- Part : xc7s75fpga484-1

로직 설계 및 합성

(1) 프로젝트 선언

원문 설명에 대응하는 실제 화면

New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts | Boards

[Reset All Filters](#)

Category: All Package: All Remaining Temperature: All Remaining
Family: **Spartan-7** Speed: All Remaining Static power: All Remaining

Search:

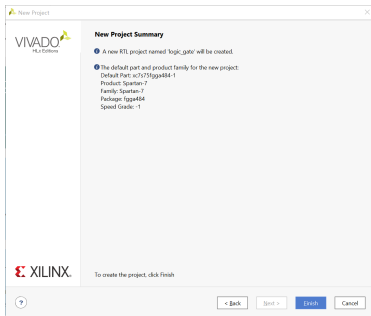
Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs
xc7s50lpg195-1Q	196	100	32600	65200	75	0
xc7s75lpg484-2	484	338	48000	96000	90	0
xc7s75lpg484-1	484	338	48000	96000	90	0
xc7s75lpg484-1L	484	338	48000	96000	90	0
xc7s75lpg484-1Q	484	338	48000	96000	90	0
xc7s75lpg484-2	676	400	48000	96000	90	0
xc7s75lpg484-1	676	400	48000	96000	90	0
xc7s75lpg484-1L	676	400	48000	96000	90	0

< ? < Back Next > Finish Cancel

로직 설계 및 합성

(1) 프로젝트 선언

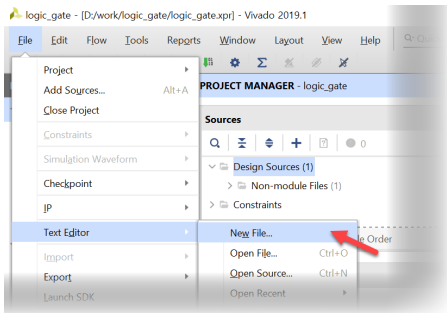
- 설정된 Summary 확인.
- Finish를 눌러 프로젝트 선언 완료



로직 설계 및 합성

(2) 로직 설계

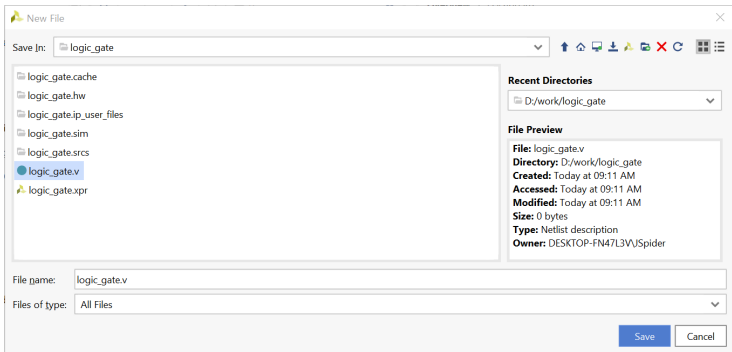
- VIVADO 프로그램
- Text Editor > New File.. 선택



로직 설계 및 합성

(2) 로직 설계

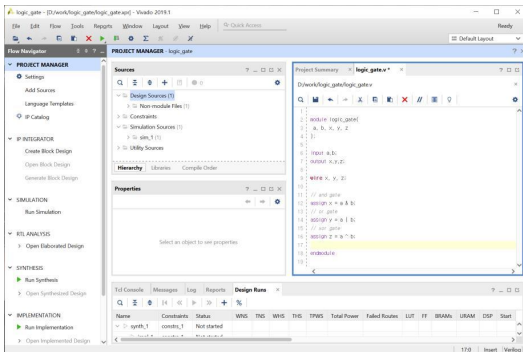
- logic_gate.v 파일로 저장



로직 설계 및 합성

(2) 로직 설계

- 로직 설계



로직 설계 및 합성

(2) 로직 설계

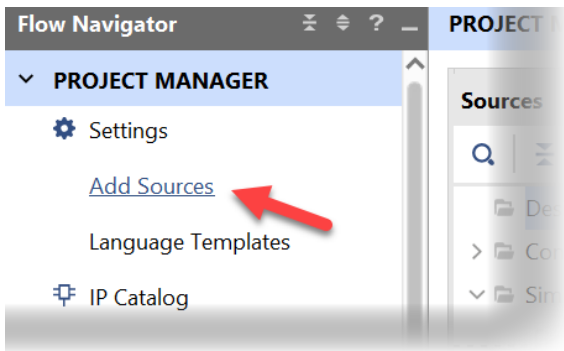
- 로직 설계

```
1 |  
2 | module logic_gate(  
3 |     a, b, x, y, z  
4 | );  
5 |  
6 | input a,b;  
7 | output x,y,z;  
8 |  
9 | wire x, y, z;  
10 |  
11 | // and gate  
12 | assign x = a & b;  
13 | // or gate  
14 | assign y = a | b;  
15 | // xor gate  
16 | assign z = a ^ b;  
17 |  
18 | endmodule  
19 |
```

로직 설계 및 합성

(2) 로직 설계

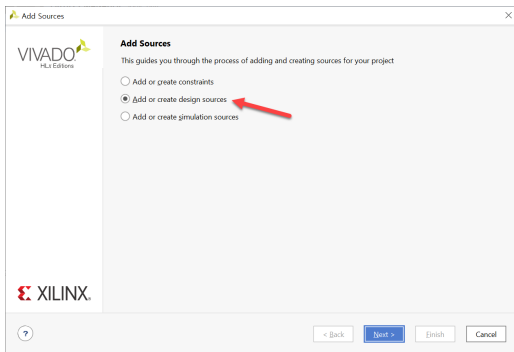
- PROJECT MANAGER > Add Sources 선택



로직 설계 및 합성

(2) 로직 설계

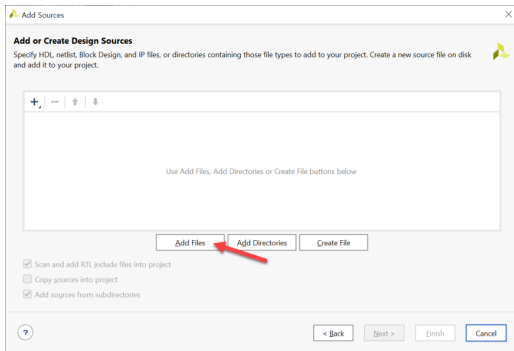
- Add Sources 창에서 Add or Create design sources 선택



로직 설계 및 합성

(2) 로직 설계

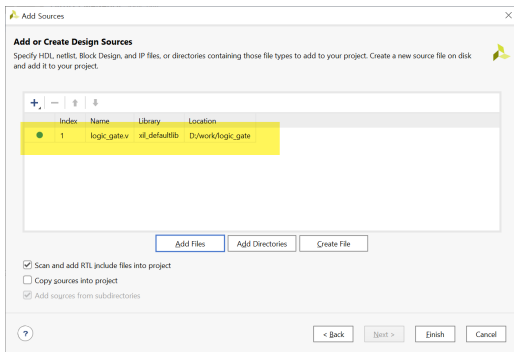
- Add Files 버튼을 눌러, 앞의 logic_gate.v 파일 추가



로직 설계 및 합성

(2) 로직 설계

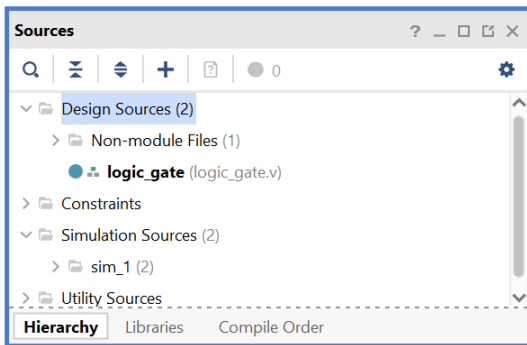
- Add Files 버튼을 눌러, 앞의 logic_gate.v 파일 추가



로직 설계 및 합성

(2) 로직 설계

- Sources 창에서 파일 추가된 것을 확인



로직 설계 및 합성

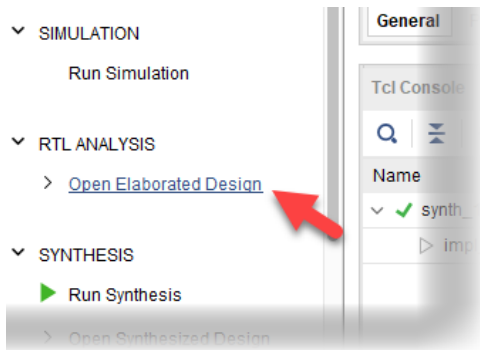
(3) Assignment

- 프로젝트의 타겟이 되는 FPGA 디바이스와 설계된 로직에서 사용하는 입출력 단자를 하드웨어와 연결하기 위하여 핀을 설정하는 것

로직 설계 및 합성

(3) Assignment

- PROJECT MANAGER의 RTL ANALYSIS 탭의 Open Elaborated Design 부분을 마우스로 클릭



로직 설계 및 합성

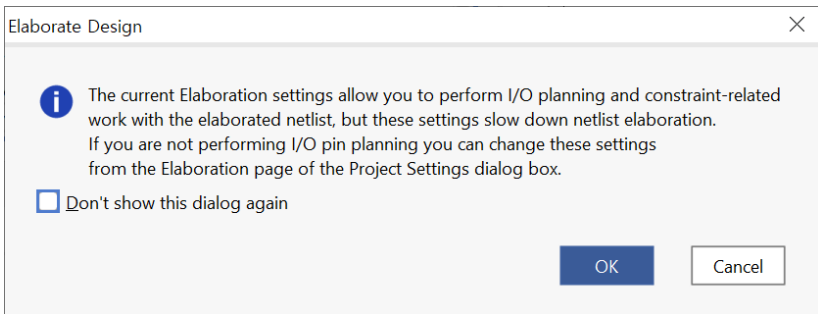
(3) Assignment

- 메시지 창이 나타난다.
- OK 버튼을 누르면 Elaboration 과정이 진행됨.
- 문법에 오류부분도 같이 체크하여 오류가 있다면 에러 메시지가 출력됨.

로직 설계 및 합성

(3) Assignment

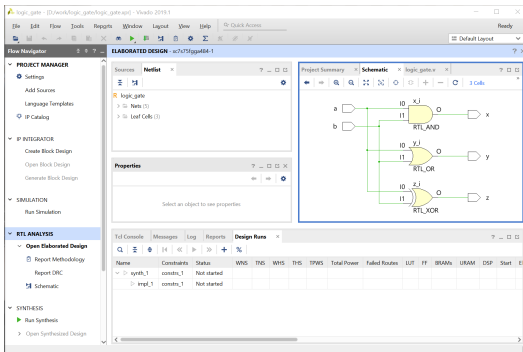
원문 설명에 대응하는 실제 화면



로직 설계 및 합성

(3) Assignment

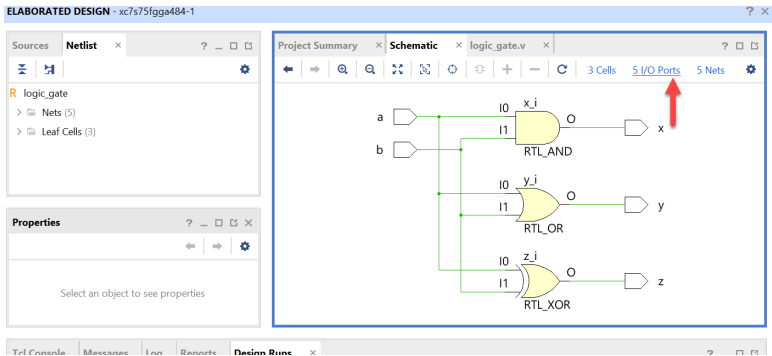
- 설계된 Verilog HDL 구문이 Schematic으로 표현되는 것을 확인



로직 설계 및 합성

(3) Assignment

- Schematic 화면 오른 쪽 윗 부분의 5 I/O Ports 선택



로직 설계 및 합성

(3) Assignment

- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

포트 이름 핀 번호			포트 이름 핀 번호		
a	K4	SW_1	x	L4	LED1
b	N8	SW_2	y	M4	LED2
			z	M2	LED3

원문 표기 유지. 실제 속성값은 **보충 설명**을 확인한다.

로직 설계 및 합성

(3) Assignment

- Package Pin 부분에 핀 설정
- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

로직 설계 및 합성

(3) Assignment

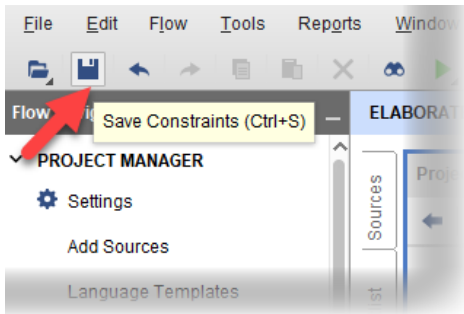
원문 설명에 대응하는 실제 화면

Tcl Console Messages Log Reports Design Runs Find Results x										
Q [Icons]										
Name	Direction	Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength
a	IN			K4	☒	35	LVC MOS33*	3.300		
b	IN			N8	☒	35	LVC MOS33*	3.300		
x	OUT			L4	☒	35	LVC MOS33*	3.300		12
y	OUT			M4	☒	35	LVC MOS33*	3.300		12
z	OUT			M2	☒	35	LVC MOS33*	3.300		12

로직 설계 및 합성

(3) Assignment

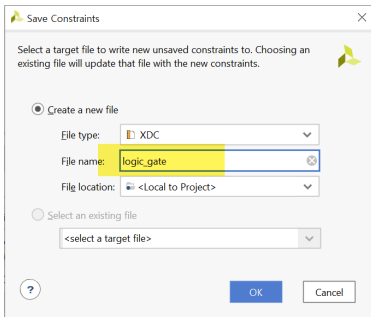
- 핀 설정 저장
- 파일명은 logic_gate로 설정



로직 설계 및 합성

(3) Assignment

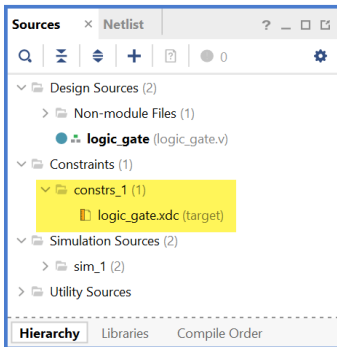
- 핀 설정 저장
- 파일명은 logic_gate로 설정



로직 설계 및 합성

(3) Assignment

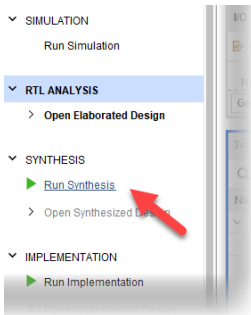
- Sources 창에 추가된 것 확인



로직 설계 및 합성

(4) Compile

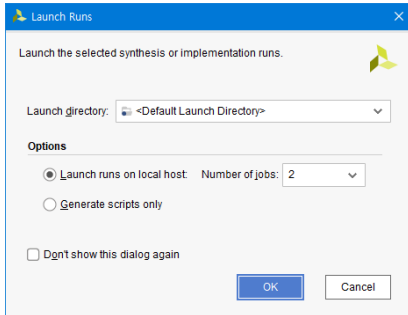
- SYNTHESIS > Run Synthesis
- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



로직 설계 및 합성

(4) Compile

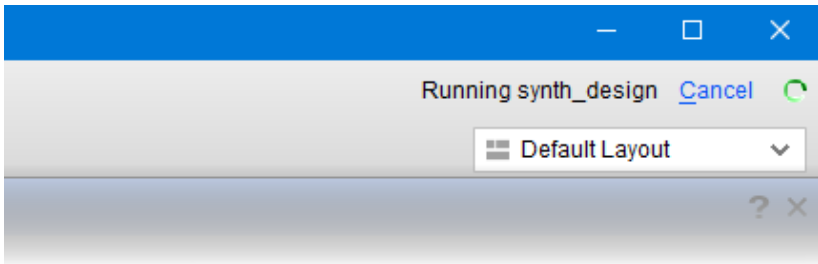
- SYNTHESIS > Run Synthesis
- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



로직 설계 및 합성

(4) Compile

- Synthesis 동작 확인 위치
- 프로그램 상단 오른쪽 회전



로직 설계 및 합성

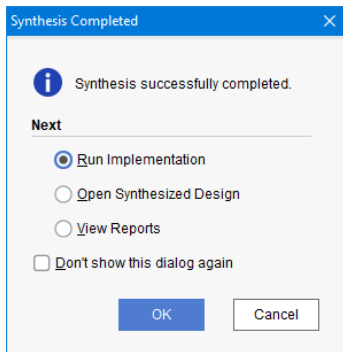
(4) Compile

- Synthesis 후 IMPLEMENTATION 진행
- Implementation
- 합성한 결과를 이용하여 실제 FPGA의 Logic Cell 단위로 바꾸어 FPGA내에 위치시키고 연결하는 Place & Route를 하는 과정

로직 설계 및 합성

(4) Compile

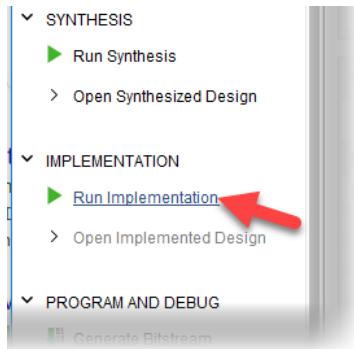
원문 설명에 대응하는 실제 화면



로직 설계 및 합성

(4) Compile

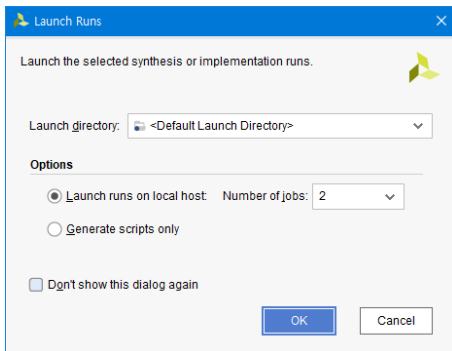
원문 설명에 대응하는 실제 화면



로직 설계 및 합성

(4) Compile

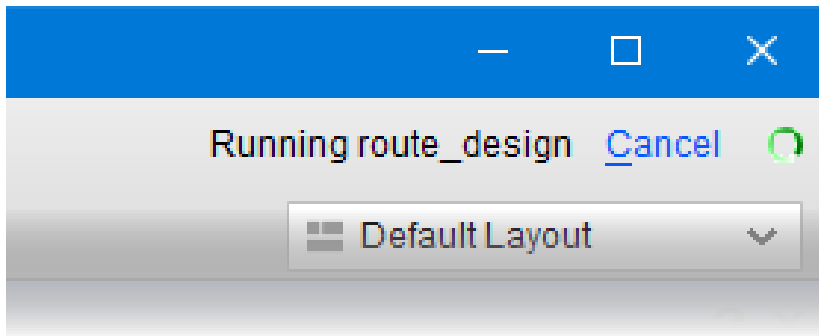
- Synthesis 후 IMPLEMENTATION 진행



로직 설계 및 합성

(4) Compile

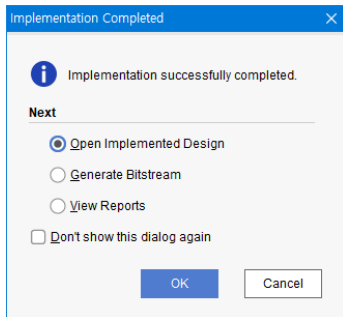
- Synthesis 후 IMPLEMENTATION 진행



로직 설계 및 합성

(4) Compile

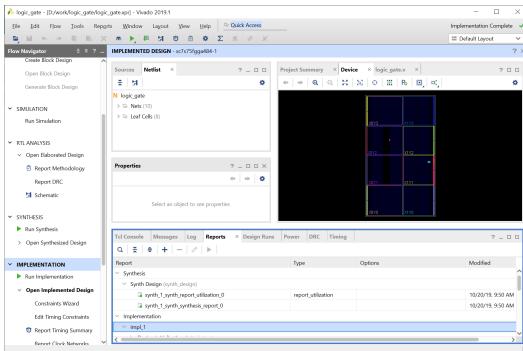
- Implementation 이 끝난 화면
- Open Implemented Design 선택 후 OK



로직 설계 및 합성

(4) Compile

- Implementation결과 확인



로직 설계 및 합성

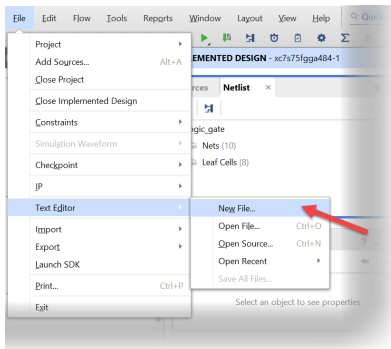
(5) Simulation

- 소프트웨어적으로 검증하는 과정
- 테스트 벤치 파일이 필요함.
- 시뮬레이션에 대한 입력 조건을 설정한 파일

로직 설계 및 합성

(5) Simulation

- File > Text Editor > New File 선택



로직 설계 및 합성

(5) Simulation

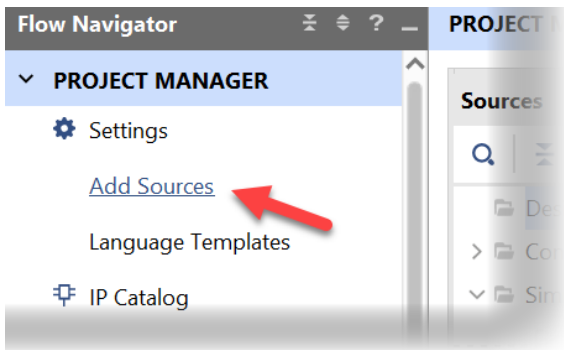
- testbench.v 파일로 저장

```
1  `timescale 10ns / 100ps
2
3  module testbench();
4      // input
5      reg a, b;
6      // output
7      wire x, y, z;
8      // Instantiate the U1
9      logic_gate u1(a, b, x, y, z);
10     // Specify input stimulus
11     initial begin
12         a = 0; b = 0;
13
14         #20 a = 1;
15         #20 a = 0; b = 1;
16         #20 a = 1;
17     end
18
19 endmodule
```

로직 설계 및 합성

(5) Simulation

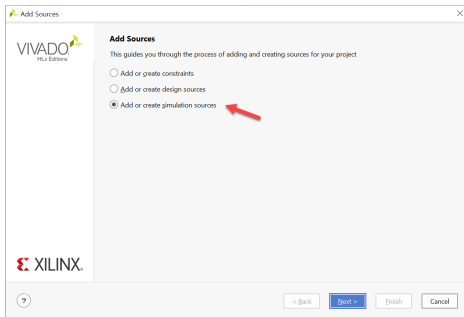
- PROJECT MANAGER Add Source 선택



로직 설계 및 합성

(5) Simulation

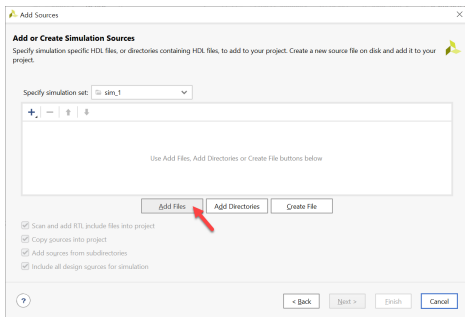
- [Add or create simulation source] 선택
- Next



로직 설계 및 합성

(5) Simulation

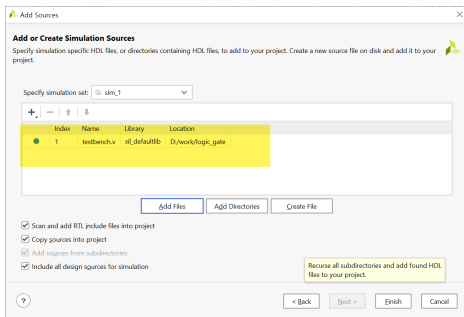
- [Add Files]을 선택
- testbench.v 파일 선택



로직 설계 및 합성

(5) Simulation

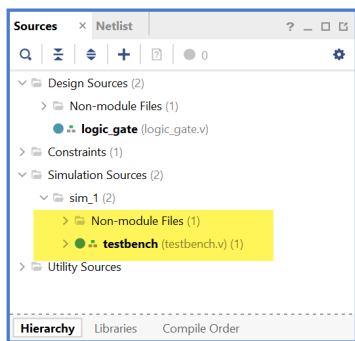
- [Add Files]을 선택
- testbench.v 파일 선택



로직 설계 및 합성

(5) Simulation

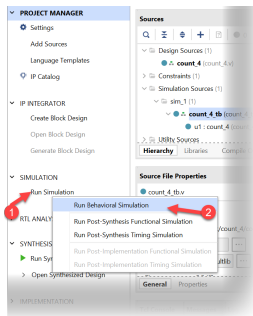
- Simulation Sources 파일 추가 확인



로직 설계 및 합성

(5) Simulation

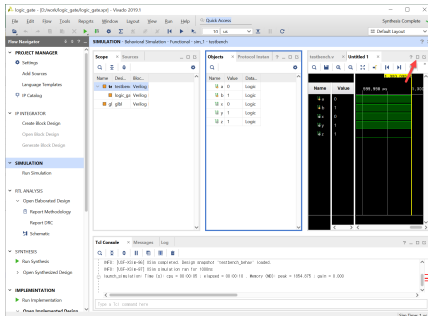
- SIMULATION > Run Simulation
- Run Behavioral Simulation 실행



로직 설계 및 합성

(5) Simulation

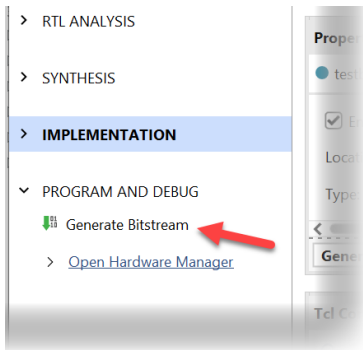
- 결과 확인
- 상단 우측의 Maximize 버튼 클릭



로직 설계 및 합성

(6) Programming

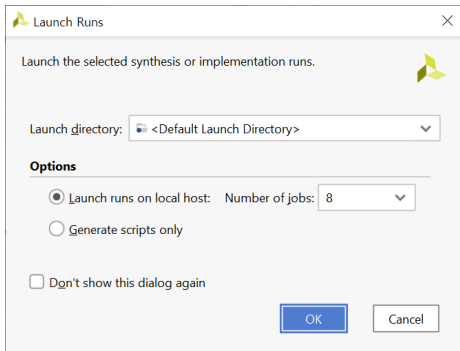
- Project Manager 창 PROGRAM AND DEBUG > Generate Bitstream



로직 설계 및 합성

(6) Programming

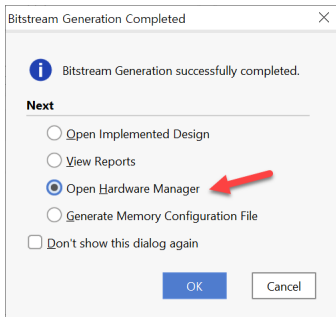
- Project Manager 창 PROGRAM AND DEBUG > Generate Bitstream



로직 설계 및 합성

(6) Programming

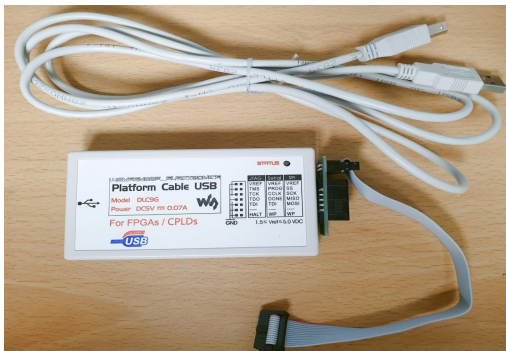
- 프로그래밍 파일 생성 완료 후
- Open Hardware Manager



로직 설계 및 합성

(6) Programming

- 하드웨어 준비



로직 설계 및 합성

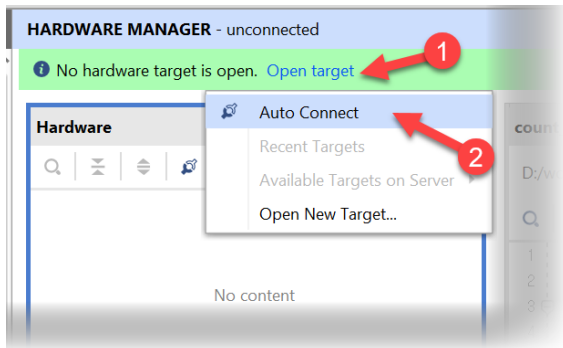
(6) Programming

- 장비의 전원이 꺼져 있는지 확인
- Xilinx Platform Cable에 USB Cable의 한 쪽 끝을 연결
- 반대쪽 끝은 PC의 USB 포트에 연결
- Xilinx Platform Cable에 연결된 2x14핀의 플랫 케이블은 장비의 Xilinx 모듈의 JTAG 포트에 연결
- 장비의 전원을 켜다.

로직 설계 및 합성

(6) Programming

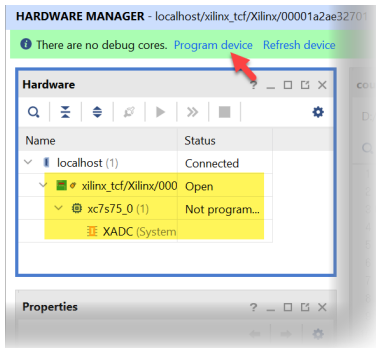
- Open Target > Auto Connect



로직 설계 및 합성

(6) Programming

- Program Device 선택



로직 설계 및 합성

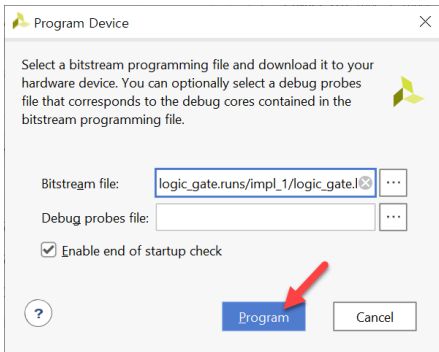
(6) Programming

- Bitstream File 선택
- <Project Folder> \ logic_gate.runs \ impl_1 \ logic_gate.bit
파일 선택
- Program 버튼 클릭

로직 설계 및 합성

(6) Programming

원문 설명에 대응하는 실제 화면



로직 설계 및 합성

(7) 동작 확인

- SW 1와 SW 2를 눌러 장비에서 어떻게 동작하는지 확인.

보충 · 원문 표기와 실행 조건

- 원본 p.24-25의 LVCM0SS33은 오타다.
실제 I/O 표준은 LVCM0S33을 선택한다.
- d:/work는 원본의 예시 작업 경로다.
실제 프로젝트를 저장한 경로를 사용한다.
- 원본 테스트벤치는 10 ns 단위에서 #20마다
입력을 바꾼다. 입력 전환 간격은 200 ns다.
- 원본의 a,b 입력 순서는 00, 10, 01, 11이다.
출력 x,y,z를 AND·OR·XOR 진리표와 비교한다.

이 페이지는 원본과 구분한 보충 설명이다. 원본 코드 이미지는 유지했다.

보충 · VS Code 사전 시뮬레이션

1. logic_gate.v와 테스트벤치를 VS Code에서 연다.
2. 배포 workspace의 02 Simulate로 자기검사 TB를 실행한다.
3. 원본 테스트벤치에는 자동 종료·VCD 저장 구문이 없다.
사전 실습은 별도 TB로 4개 입력·40 ns와 VCD를 검사한다.
4. 생성한 파형에서 a,b,x,y,z를 확인하고 예상값과 비교한다.
5. 실행 방법·코드 설명·파형 해석을 실험 전 레포트에 쓴다.

VS Code 환경설정 매뉴얼 · [레포트 안내](#)

사전 단계는 이번 수업의 보충 안내이며 원본 Vivado 화면과 구분한다.

보충 · 보드 결과와 실험 후 레포트

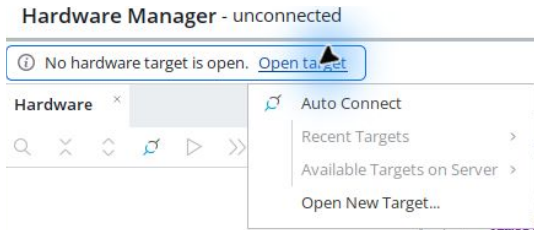
1. Vivado 파형을 VS Code 사전 시뮬레이션과 비교한다.
2. 생성한 logic_gate.bit를 실제 FPGA에 기록한다.
3. SW_1·SW_2의 네 입력 조합과 LED1·LED2·LED3를 확인한다.
4. 연결 상태는 사진, 입력에 따른 출력 변화는 영상으로 남긴다.
5. 코드·파형·사진·영상을 GitHub에 올리고 레포트에 연결한다.

실제 결과와 예상값을 비교하고 오류·수정 사항을 설명한다.

원본 화면의 성공 표시와 본인의 실행 결과를 구분한다.

이 실습 목차 · 전체 목차 PDF

실제 보드 연결



Open Hardware Manager → Open target → Auto Connect.

KEY1=a, KEY2=b. LED1=x, LED2=y, LED3=z.

장치가 없으면 보드 전원·USB JTAG·드라이버·VM USB 연결을 점검한다.

Program Device와 동작 확인

1. 연결된 장치 이름이 xc7s75인지 확인한다.
2. 장치 우클릭 → Program Device.
3. 이번 프로젝트의 logic_gate.bit를 선택하고 Program.
4. 기록 완료를 확인한 뒤 입력을 바꾸고 출력과 예상값을 비교한다.
5. 기록 성공 화면과 실제 보드 동작은 각각 증빙한다.

제작 환경의 실제 보드 기록·촬영 검증 여부는 검증표를 따른다. 파일 생성만으로 보드 동작 성공을 선언하지 않는다.

사진과 시연 영상 촬영

KEY1=a, KEY2=b. LED1=x, LED2=y, LED3=z.

사진에는 보드 연결과 입력·출력 위치가 함께 보이게 한다.

영상에는 입력을 조작하는 과정과 그에 따른 출력 변화를 담는다.

예상표의 정상·경계 조건을 직접 보여주고 회로 번호를 파일명에 적는다.

통합 영상이라면 각 회로의 타임스탬프를 레포트에서 연결한다.

GitHub 결과 정리

1. reports/pre와 reports/post에 실험 전·후 레포트를 둔다.
 2. evidence/vscode와 evidence/vivado에 로그·파형·캡처를 구분한다.
 3. evidence/board/photos와 videos에 직접 촬영한 자료를 둔다.
 4. 큰 영상·bit는 배포 위치를 정하고 README와 레포트에서 연결한다.
 5. 웹에서 사진 표시와 영상 재생 또는 다운로드가 되는지 확인한다.
- 레포트 양식·예시·GitHub 안내

실험 후 레포트

1. Vivado 버전·part·top·핀 제약·코드 커밋을 기록한다.
2. VS Code와 Vivado의 입력·출력·시간·검사 수를 비교한다.
3. 합성·구현·bit 경로와 경고·수정 사항을 설명한다.
4. 장치 기록 화면·사진·영상과 해당 조건을 연결한다.
5. 예상값·두 시뮬레이션·실측의 일치 또는 차이 원인을 해석한다.
6. 미수행 항목은 미완료로 남기고 후속 확인을 적는다.

전체 목차 PDF 프로젝트로 돌아가기